

Container Congestion Predictor & Port Operations Optimiser

Hackathon Project Documentation

Problem Statement: L1 — Container Congestion Predictor & Port Operations Optimiser

The challenge asks for a solution that predicts congestion hotspots, recommends alternate routing strategies, optimises berth and crane assignments, and generates a 72-hour port operations plan for shift supervisors.

Prototype note: The current C program is a simplified prototype. It demonstrates vessel input, total-container calculation, congestion classification, and a basic alternate-route recommendation. It does not yet implement live schedules, real berth/crane optimisation, geographic routing, or a true 72-hour scheduling engine.

1. Introduction

PortMind is a prototype concept for helping port operators identify possible congestion from vessel and container information. The project converts basic vessel data into a simple congestion status and demonstrates how an alternate port could be considered when congestion is high.

2. Problem Statement

Port operators may need to coordinate many vessels, berths, cranes and yard resources. The L1 challenge focuses on predicting congestion hotspots and supporting better operational decisions before delays become severe.

3. Proposed Solution

The proposed system accepts vessel details such as vessel name, number of containers and arrival time. It calculates the total container load, classifies congestion as low, medium or high using prototype thresholds, and displays an alternate-route recommendation when congestion is high.

4. Objectives

- Predict possible congestion from available vessel information.
- Demonstrate an alternate routing decision.
- Organise vessel information clearly.
- Provide a foundation for future berth, crane and 72-hour planning modules.

5. Technology Used

Programming Language: C

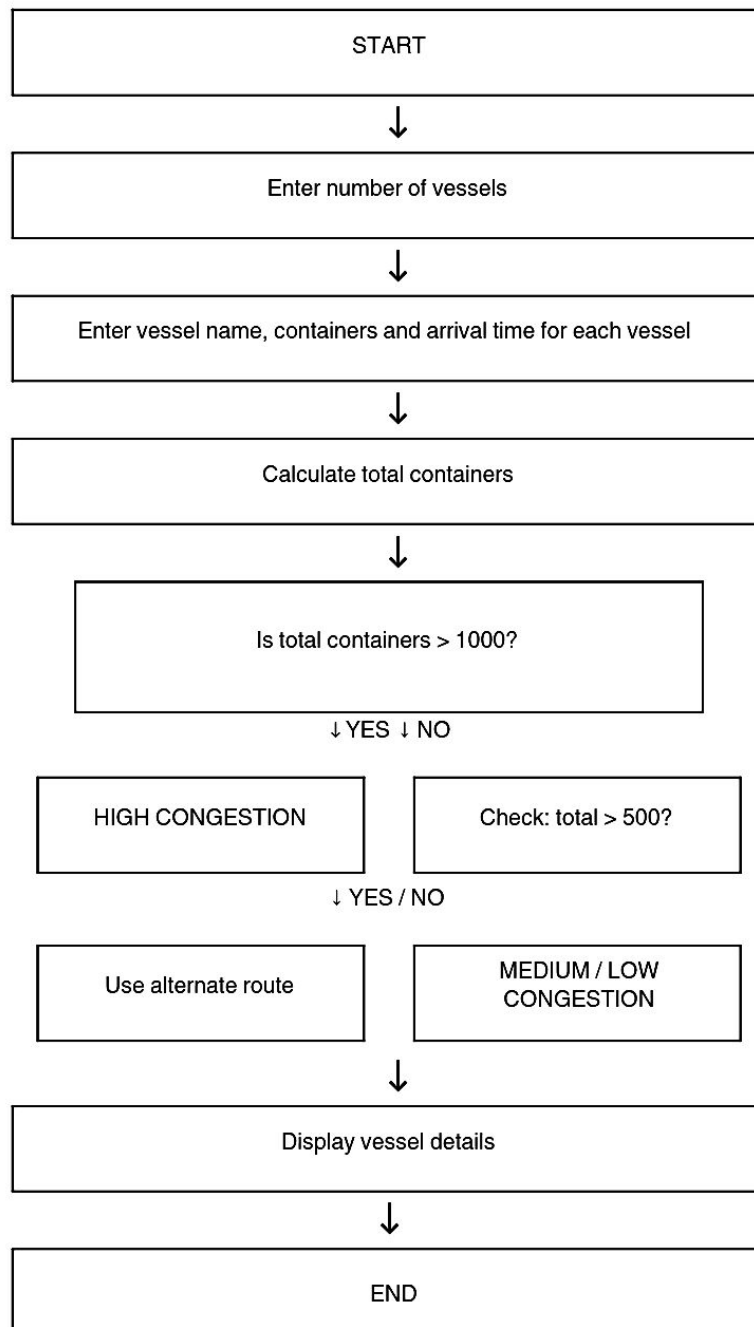
Data Structure: Structure (struct Vessel) and array

Input/Output: Standard C console input and output

Prototype environment: Any standard C compiler such as GCC or Code::Blocks.

6. System Flowchart

The following flowchart represents the logic implemented by the current C prototype.



7. Algorithm

1. Read the number of vessels.
2. Store vessel name, container count and arrival time in a structure array.
3. Add all container counts to obtain total containers.
4. If total containers are greater than 1000, classify as HIGH CONGESTION and recommend an alternate route.
5. If total containers are greater than 500 but not greater than 1000, classify as MEDIUM CONGESTION.
6. Otherwise classify as LOW CONGESTION.
7. Display all vessel details.

8. Source Code

```
#include

struct Vessel
{
    char name[50];
    int containers;
    int arrivalTime;
};

int main()
{
    struct Vessel ship[50];
    int n, i;
    int totalContainers = 0;

    printf("Enter number of vessels: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("\nEnter vessel %d name: ", i + 1);
        scanf("%s", ship[i].name);

        printf("Enter number of containers: ");
        scanf("%d", &ship[i].containers);

        printf("Enter arrival time: ");
        scanf("%d", &ship[i].arrivalTime);

        totalContainers = totalContainers + ship[i].containers;
    }

    printf("\nTotal containers = %d\n", totalContainers);

    if(totalContainers > 1000)
    {
        printf("HIGH CONGESTION\n");
        printf("Use alternate route.\n");
    }
    else if(totalContainers > 500)
    {
        printf("MEDIUM CONGESTION\n");
    }
    else
    {
        printf("LOW CONGESTION\n");
    }
}
```

```
}

printf("\n===== VESSEL DETAILS =====\n");

for(i = 0; i < n; i++)
{
printf("\nVessel Name: %s", ship[i].name);
printf("\nContainers: %d", ship[i].containers);
printf("\nArrival Time: %d\n", ship[i].arrivalTime);
}

return 0;
}
```

9. Sample Test Case

Vessel	Containers	Arrival Time
MAERSK	500	10
MSC	300	15
CMA	400	20
Total	1200	

Expected result: Total containers = 1200 → HIGH CONGESTION → Use alternate route.

Important: The 500 and 1000 container thresholds are prototype values chosen for demonstration. They are not specified in the L1 challenge statement.

10. Alternate Route Concept

If the primary destination port becomes highly congested, a future version of PortMind can compare candidate ports using congestion level, available capacity, estimated travel time and operational constraints, then recommend a suitable alternative. The current C prototype only prints a basic alternate-route message; it does not calculate real geographic routes.

11. Limitations

The prototype does not yet use live vessel schedules, actual berth capacities, real crane availability, geographic routing data, weather/historical data, a database, a GUI dashboard, or a true 72-hour scheduling engine.

12. Future Scope

Future versions can add a dashboard, CSV/database input, congestion scoring from real port data, berth and crane optimisation, alternate-port comparison, historical/ML prediction, and an automated 72-hour operational plan.

13. Conclusion

PortMind demonstrates the core idea of using vessel information to identify potential port congestion and support an alternate-route decision. It provides a simple foundation that can be expanded into the full L1 hackathon solution.